

Projeto Acapra

FrontEnd

Sistema de adoção para Pets resgatados

Equipe de Desenvolvimento:

João Vitor Pereira

Leonardo Tachini

Luis Felipe Barnabe

Luis Felipe Peirão

Maria de Fatima Groh

Sumário

1 - Visão geral.....	2
2 - Stack principal.....	2
3 - Estrutura de pastas.....	3
4 - Execução do projeto (front-end).....	4
5 - Código principal.....	5
6 - Integração com API.....	6
7 - Rotas principais.....	6
8 - Fluxo de dados (UI -> API).....	7
9 - Build de produção.....	7
10 - Conclusão.....	8

1) Visão geral

O frontend do Acapra foi desenvolvido em React + TypeScript, utilizando Vite como bundler e ambiente de desenvolvimento. Ele se comunica com a API REST do backend para exibir dados, permitir interações e conduzir o fluxo de adoção.

2) Stack principal

- Framework: React 18+
- Linguagem: TypeScript
- Build Tool: Vite
- Estilo: CSS (global e modular)
- Lint: ESLint (`eslint.config.js`)
- Gerenciador de pacotes: npm

3) Estrutura de pastas

- .github/workflows
- build/types
- database_models
- public
- src
 - components
 - features
 - services
 - models
 - routes
 - styles
 - main.tsx
 - App.tsx
- index.html
- package.json
- vite.config.ts
- eslint.config.js
- tsconfig*.json

4) Execução do projeto (front-end)

1. Instalar dependências

npm install

2. Configurar ambiente (copiar exemplo e editar base URL da API)

cp .env.example .env

3. Rodar ambiente de desenvolvimento

npm run dev

A aplicação roda em `http://localhost:...`

5) Código principal

- `src/main.tsx / src/App.tsx`

- Responsáveis por montar o React e iniciar o roteamento da aplicação.
- Importam estilos globais e definem o layout raiz.

- `src/routes/`

- Define rotas públicas e privadas.
- Exemplo esperado de estrutura:


```
<Route path="/" element={<Home />} />
<Route path="/animals" element={<AnimalList />} />
<Route path="/animals/:id" element={<AnimalDetail />} />
<PrivateRoute path="/adoptions" element={<AdoptionFlow />} />
```

- `src/services/api.ts`

- Centraliza requisições HTTP (axios ou fetch).
- Usa `import.meta.env.VITE_API_BASE_URL` como base.
- Exemplo simplificado:


```
import axios from 'axios';
export const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
});
```

- `src/features/animals/`

- Contém telas e hooks relacionados ao módulo **Animais**.
- Exemplo: `useAnimals.ts` faz requisições à API e controla estados (loading, error).


```
const { data, isLoading, error } = useQuery(['animals'], () => api.get('/animals'));
```

- `src/models/`

- Define tipos compartilhados (DTOs) para comunicação com o backend.


```
export interface Animal {
  id: number;
  nome: string;
  especie: string;
  idade: number;}
```

- `src/components/`

- Componentes reutilizáveis: botões, cards, formulários, modais.
- Padronizam estilo e comportamento entre telas.

6) Integração com API

- Comunicação RESTful com backend Acapra.
- Cada módulo (animals, adoptions, users) tem seus próprios serviços.
- Requisições usam baseURL configurada no .env.
- Tratamento de erros centralizado (exibição de mensagens, redirecionamentos).

7) Rotas principais

Rota	Página	Protegida	Descrição
/	Home	✗	Página inicial
/login	Login	✗	Autenticação de usuário
/animals	Lista de animais	✓	Mostra todos os animais disponíveis
/animals/:id	Detalhe	✓	Informações detalhadas de um animal
/adoptions	Adoções	✓	Fluxo de solicitação de adoção
/admin/*	Painel administrativo	✓	Gerenciamento interno

8) Fluxo de dados (UI -> API)

sequenceDiagram

participant UI as Página React

participant API as Service HTTP

participant BE as Backend

UI->>API: GET /animals

API->>BE: requisição para endpoint REST

BE-->>API: resposta JSON

API-->>UI: retorna dados tipados

UI-->>UI: renderiza lista de animais

9) Build de produção

npm run build

npm run preview

- A build é gerada na pasta /dist.
- Pode ser servida via qualquer servidor estático (Nginx, Vercel, etc.).

10) Conclusão

O front-end do projeto Acapra é a camada responsável por toda a interação com o usuário, permitindo o acesso às informações de animais, adoções e perfis de forma intuitiva e organizada. Sua estrutura modular, desenvolvida em React e TypeScript, favorece a manutenção, expansão e reutilização de componentes.

A arquitetura foi pensada para separar de forma clara as responsabilidades: componentes cuidam da interface e experiência visual, serviços fazem a comunicação com a API, e models garantem consistência nos dados trafegados entre o front e o backend. Essa abordagem facilita a evolução do sistema sem comprometer outras partes do código.

Com a configuração simples do ambiente (`npm install` e `npm run dev`), o projeto pode ser executado rapidamente em qualquer máquina. A utilização de variáveis de ambiente garante flexibilidade para conectar diferentes instâncias do backend.

Em conjunto com o backend, o front-end do Acapra forma uma aplicação completa e funcional voltada para o gerenciamento e acompanhamento de processos de adoção, oferecendo uma base sólida para futuras melhorias e novas funcionalidades.